

Giới thiệu về pandas

Contents

- 6.1. Các cấu trúc dữ liệu cơ bản của pandas
- 6.2. Chỉ số trong các cấu trúc dữ liệu cơ bản
- 6.3. Tính toán trên các cấu trúc dữ liệu cơ bản
- 6.4. Các phương thức nhằm tóm tắt và tính toán thống kê mô tả



KHOA HỌC DỮ LIỆU TRONG KINH TẾ VÀ KINH DOANH

`pandas` là công cụ không thể thiếu của bất kỳ nhà phân tích dữ liệu hoặc nhà khoa học dữ liệu nào khi sử dụng Python. Thư viện này cung cấp các cấu trúc dữ liệu hiệu suất cao để sử dụng và các công cụ phân tích dữ liệu tiên tiến. Cái tên `pandas` bắt nguồn từ “panel data” (dữ liệu bảng), một thuật ngữ kinh tế lượng cho các bộ dữ liệu đa chiều, có cấu trúc, và từ “Python data analysis”.

Trong chương này, chúng tôi sẽ tập trung vào hai cấu trúc dữ liệu chính của `pandas`, đó là `Series`, hay còn gọi là *chuỗi* và `DataFrame`, hay còn gọi là *dữ liệu kiểu bảng*. Mặc dù chúng không phải là giải pháp phổ quát cho mọi vấn đề về thao tác dữ liệu chuyên sâu, nhưng chúng thường là công cụ phù hợp và hiệu quả cho hầu hết các ứng dụng. `pandas` mượn nhiều ý tưởng về lập trình mảng từ `NumPy`, nhưng được thiết kế để làm việc với dữ liệu dạng bảng dễ dàng hơn. Ngược lại, `Numpy` phù hợp nhất để làm việc với dữ liệu số đồng nhất được tổ chức trong các mảng. Bạn đọc sẽ thấy rằng, `pandas` và `Numpy` hoạt động tốt cùng nhau. Thông thường, dữ liệu được thao tác trong `pandas` sau đó được chuyển vào các mảng `Numpy` để áp dụng các công cụ xây dựng mô hình trong `scikit-learn`.

Trong nhiều năm, `pandas` đã phát triển để xử lý các trường hợp dữ liệu sử dụng ngày càng phức tạp. Chúng tôi sẽ cố gắng tập trung vào các tính năng quan trọng nhất, làm sáng tỏ những điều

cơ bản trước khi xem xét các mô hình sử dụng dữ liệu phức tạp hơn. Chúng tôi hy vọng rằng cuốn sách này sẽ cung cấp cho bạn nền tảng để tự tin giải quyết các vấn đề về dữ liệu thực tế.

Thông thường, một trong những quy ước gọi thư viện `pandas` đầu tiên bạn sẽ thấy là:

```
import pandas as pd
```

Do đó, bất cứ khi nào bạn thấy `pd.` thì đó là đang đề cập đến pandas.

Chương này sẽ giới thiệu cho bạn các cấu trúc dữ liệu cơ bản trong pandas và cách tương tác với chúng.

6.1. Các cấu trúc dữ liệu cơ bản của pandas

Để bắt đầu với `pandas`, bạn sẽ cần làm quen với hai cấu trúc dữ liệu cơ bản của nó: `Series` và `DataFrame`.

6.1.1. Series (chuỗi)

Một chuỗi, hay `Series`, là một đối tượng giống mảng một chiều chứa một chuỗi các giá trị và một mảng các nhãn dữ liệu (hay tên dữ liệu) liên kết với các giá trị. Các nhãn dữ liệu còn được gọi là *chỉ số*, hay *index*, của nó. `Series` đơn giản nhất được hình thành chỉ từ một mảng dữ liệu:

```
import numpy as np
import pandas as pd

obj = pd.Series([4, 7, -5, 3])
obj
```

```
0    4
1    7
2   -5
3    3
dtype: int64
```

Biểu diễn chuỗi của một `Series` được hiển thị cho thấy chỉ số ở bên trái và các giá trị ở bên phải. Vì chúng ta không chỉ định chỉ số cho dữ liệu, một chỉ số mặc định bao gồm các số nguyên

từ 0 đến `n - 1`, được tạo ra, với `n` là độ dài của dữ liệu. Bạn có thể trích xuất biểu diễn mảng của giá trị và chỉ số của `Series` thông qua các thuộc tính `array` và `index`:

```
obj.array
```

```
<NumpyExtensionArray>  
[np.int64(4), np.int64(7), np.int64(-5), np.int64(3)]  
Length: 4, dtype: int64
```

```
obj.index
```

```
RangeIndex(start=0, stop=4, step=1)
```

Thông thường, bạn có thể tạo một `Series` với một chỉ số xác định từng điểm dữ liệu bằng một nhãn:

```
obj2 = pd.Series([4, 7, -5, 3], index=["d", "b", "a", "c"])  
obj2
```

```
d    4  
b    7  
a   -5  
c    3  
dtype: int64
```

```
obj2.index
```

```
Index(['d', 'b', 'a', 'c'], dtype='object')
```

So với một mảng trong `NumPy`, bạn có thể sử dụng các tên thay cho chỉ số khi chọn các giá trị đơn lẻ hoặc một tập hợp các giá trị:

```
obj2["a"]
```

```
np.int64(-5)
```

```
obj2["d"] = 6  
obj2[["c", "a", "d"]]
```

```
c      3  
a     -5  
d      6  
dtype: int64
```

Ở trên `["c", "a", "d"]` được hiểu là một danh sách các chỉ số, ngay cả khi không chứa số nguyên.

Chuỗi của `pandas` có thể sử dụng các hàm `Numpy` hoặc các phép toán giống như `Numpy`, chẳng hạn như lọc với một mảng logical, các phép toán số học vô hướng, hoặc áp dụng các hàm toán học, và kết quả trả lại sẽ bảo toàn liên kết giữa chỉ số với giá trị:

```
import numpy as np  
obj2[obj2 > 0]
```

```
d      6  
b      7  
c      3  
dtype: int64
```

```
obj2 * 2
```

```
d     12  
b     14  
a    -10  
c      6  
dtype: int64
```

```
np.exp(obj2)
```

```
d      403.428793
b     1096.633158
a       0.006738
c      20.085537
dtype: float64
```

Một cách khác để hình dung về `Series` của `pandas` là một từ điển có thứ tự cố định. Trong nhiều ngữ cảnh mà bạn có thể sử dụng như một `dict`:

```
"b" in obj2
```

```
True
```

```
"e" in obj2
```

```
False
```

Nếu bạn có dữ liệu được chứa trong một `dict` của Python, bạn có thể tạo một `Series` bằng hàm `Series`

```
sdata = {"Ohio": 35000, "Texas": 71000, "Oregon": 16000, "Utah": 5000}
obj3 = pd.Series(sdata)
obj3
```

```
Ohio      35000
Texas     71000
Oregon    16000
Utah       5000
dtype: int64
```

Khi bạn chỉ sử dụng một `dict`, chỉ số trong `Series` kết quả sẽ là các khóa của `dict` theo thứ tự. Bạn có thể ghi đè lên chỉ số bằng cách truyền các khóa `dict` theo thứ tự bạn muốn chúng xuất hiện trong `Series` kết quả:

```
states = ["California", "Ohio", "Oregon", "Texas"]
obj4 = pd.Series(sdata, index=states)
obj4
```

```
California      NaN
Ohio            35000.0
Oregon          16000.0
Texas           71000.0
dtype: float64
```

Ở trên có ba giá trị được tìm thấy trong `sdata` đã được đặt vào các vị trí thích hợp. Vì không tìm thấy giá trị nào cho `"California"`, nên giá trị xuất hiện tương ứng với chỉ số này là `NaN`. Vì chỉ số `"Utah"` không được bao gồm trong `states`, nó bị loại khỏi đối tượng kết quả.

Cuốn sách này sẽ sử dụng thuật ngữ "missing value" hoặc "NA" để chỉ dữ liệu không quan sát được. Các hàm `isna` và `notna` trong `pandas` được sử dụng để phát hiện dữ liệu bị thiếu:

```
pd.isna(obj4)
```

```
California      True
Ohio            False
Oregon          False
Texas           False
dtype: bool
```

```
pd.notna(obj4)
```

```
California      False
Ohio            True
Oregon          True
Texas           True
dtype: bool
```

`Series` cũng có các phương thức `.isna()` để xác định giá trị không quan sát được:

```
obj4.isna()
```

```
California      True
Ohio            False
Oregon          False
Texas           False
dtype: bool
```

Một tính năng hữu ích của `Series` cho nhiều ứng dụng là đối tượng này **tự động căn chỉnh theo chỉ số** trong các phép toán số học. Bạn đọc quan sát ví dụ sau:

```
obj3
```

```
Ohio      35000
Texas     71000
Oregon    16000
Utah       5000
dtype: int64
```

```
obj4
```

```
California    NaN
Ohio          35000.0
Oregon        16000.0
Texas         71000.0
dtype: float64
```

```
obj3 + obj4
```

```
California    NaN
Ohio          70000.0
Oregon        32000.0
Texas        142000.0
Utah           NaN
dtype: float64
```

Bạn đọc có thể thấy rằng các chỉ số đã được tự động khớp với nhau trước khi thực hiện phép cộng. Việc tự động căn chỉnh chỉ số sẽ được thảo luận chi tiết hơn trong các phần sau.

Cả đối tượng `Series` và chỉ số của đối tượng này đều có thuộc tính `name`, được tích hợp với các chức năng khác của `pandas`:

```
obj4.name = "population"
obj4.index.name = "state"
obj4
```

```
state
California      NaN
Ohio            35000.0
Oregon          16000.0
Texas           71000.0
Name: population, dtype: float64
```

Chỉ số của một `Series` có thể được thay đổi trực tiếp bằng cách gán:

```
obj
```

```
0    4
1    7
2   -5
3    3
dtype: int64
```

```
obj.index = ["Bob", "Steve", "Jeff", "Ryan"]
obj
```

```
Bob      4
Steve    7
Jeff    -5
Ryan     3
dtype: int64
```

6.1.2. DataFrame

`DataFrame` biểu diễn dữ liệu dạng bảng chữ nhật và chứa một tập hợp các cột có thứ tự, mỗi cột có thể là một kiểu giá trị khác nhau. `DataFrame` có cả chỉ số hàng và chỉ số cột. Cũng có thể hiểu `DataFrame` là một từ điển (`dict`) của các chuỗi (`Series`) có cùng một mảng chỉ số.

Có nhiều cách để xây dựng một `DataFrame`, mặc dù một trong những cách phổ biến nhất là từ một từ điển (`dict`) của các danh sách (`list`) có độ dài bằng nhau:


```
data = {"state": ["Ohio", "Ohio", "Ohio", "Nevada", "Nevada", "Nevada"],
        "year": [2000, 2001, 2002, 2001, 2002, 2003],
        "pop": [1.5, 1.7, 3.6, 2.4, 2.9, 3.2]}
frame = pd.DataFrame(data)
frame
```

	state	year	pop
0	Ohio	2000	1.5
1	Ohio	2001	1.7
2	Ohio	2002	3.6
3	Nevada	2001	2.4
4	Nevada	2002	2.9
5	Nevada	2003	3.2

`DataFrame` kết quả sẽ có chỉ số được gán tự động như với `Series`, và các cột được đặt theo thứ tự của các khóa của từ điển.

Đối với các `DataFrame` lớn, phương thức `head` được sử dụng để lựa chọn các hàng đầu tiên:

```
frame.head()
```

	state	year	pop
0	Ohio	2000	1.5
1	Ohio	2001	1.7
2	Ohio	2002	3.6
3	Nevada	2001	2.4
4	Nevada	2002	2.9

Tương tự, phương thức `tail` được sử dụng để lựa chọn các hàng dưới cùng:

```
frame.tail()
```

	state	year	pop
1	Ohio	2001	1.7
2	Ohio	2002	3.6
3	Nevada	2001	2.4
4	Nevada	2002	2.9
5	Nevada	2003	3.2

Bạn có thể sắp xếp lại thứ tự các cột bằng cách sử dụng tham số `columns`

```
pd.DataFrame(data, columns=["year", "state", "pop"])
```

	year	state	pop
0	2000	Ohio	1.5
1	2001	Ohio	1.7
2	2002	Ohio	3.6
3	2001	Nevada	2.4
4	2002	Nevada	2.9
5	2003	Nevada	3.2

Nếu bạn tạo `DataFrame` với một cột không có trong `dict`, giá trị cột đó sẽ chỉ bao gồm các giá trị `NA`

```
frame2 = pd.DataFrame(data, columns=["year", "state", "pop", "debt"])
frame2
```

	year	state	pop	debt
0	2000	Ohio	1.5	NaN
1	2001	Ohio	1.7	NaN
2	2002	Ohio	3.6	NaN
3	2001	Nevada	2.4	NaN
4	2002	Nevada	2.9	NaN
5	2003	Nevada	3.2	NaN

```
frame2.columns
```

```
Index(['year', 'state', 'pop', 'debt'], dtype='object')
```

Một cột trong `DataFrame` có thể được truy xuất dưới dạng `Series` bằng cách sử dụng ký hiệu kiểu `dict` hoặc bằng cách gọi như một thuộc tính:

```
frame2["state"] # gọi tên
```

```
0    Ohio
1    Ohio
2    Ohio
3  Nevada
4  Nevada
5  Nevada
Name: state, dtype: object
```

```
frame2.year # gọi theo kiểu thuộc tính
```

```
0    2000
1    2001
2    2002
3    2001
4    2002
5    2003
Name: year, dtype: int64
```

Mỗi cột kết quả được trả về là `Series` có cùng chỉ số với `DataFrame`.

Các hàng của `DataFrame` cũng có thể được truy xuất theo vị trí hoặc tên với thuộc tính đặc biệt `loc`:

```
frame2.loc[1]
```

```
year    2001
state   Ohio
pop     1.7
debt    NaN
Name: 1, dtype: object
```

```
frame2.loc[[0, 1]]
```

	year	state	pop	debt
0	2000	Ohio	1.5	NaN
1	2001	Ohio	1.7	NaN

Các cột của `DataFrame` có thể được sửa đổi bằng cách gán giá trị một cách trực tiếp. Ví dụ, cột `"debt"` trống có thể được gán một giá trị vô hướng hoặc một mảng các giá trị:

```
frame2["debt"] = 11.5
frame2
```

	year	state	pop	debt
0	2000	Ohio	1.5	11.5
1	2001	Ohio	1.7	11.5
2	2002	Ohio	3.6	11.5
3	2001	Nevada	2.4	11.5
4	2002	Nevada	2.9	11.5
5	2003	Nevada	3.2	11.5

```
frame2["debt"] = np.arange(6.0)
frame2
```

	year	state	pop	debt
0	2000	Ohio	1.5	0.0
1	2001	Ohio	1.7	1.0
2	2002	Ohio	3.6	2.0
3	2001	Nevada	2.4	3.0
4	2002	Nevada	2.9	4.0
5	2003	Nevada	3.2	5.0

Khi bạn đọc sử dụng một danh sách hoặc một mảng để gán cho một cột, độ dài phải khớp với độ dài của `DataFrame`. Nếu bạn gán một `Series`, các chỉ số sẽ được **tự động căn chỉnh** theo chỉ số của `DataFrame`, chèn các giá trị bị thiếu vào bất kỳ lỗ hổng nào:

```
val = pd.Series([-1.2, -1.5, -1.7], index=[0, "four", "five"])
frame2["debt"] = val
frame2
```

	year	state	pop	debt
0	2000	Ohio	1.5	-1.2
1	2001	Ohio	1.7	NaN
2	2002	Ohio	3.6	NaN
3	2001	Nevada	2.4	NaN
4	2002	Nevada	2.9	NaN
5	2003	Nevada	3.2	NaN

Việc gán một cột không tồn tại trong `DataFrame` sẽ tự động tạo ra một cột mới. Từ khóa `del` được sử dụng để xóa các cột. Bạn đọc quan sát ví dụ sau, khi chúng ta thêm một cột mới có tên `eastern` chứa dữ liệu kiểu logical sau đó sử dụng từ khóa `del` để xóa cột này:

```
frame2["eastern"] = frame2["state"] == "Ohio"
frame2
```

	year	state	pop	debt	eastern
0	2000	Ohio	1.5	-1.2	True
1	2001	Ohio	1.7	NaN	True
2	2002	Ohio	3.6	NaN	True
3	2001	Nevada	2.4	NaN	False
4	2002	Nevada	2.9	NaN	False
5	2003	Nevada	3.2	NaN	False

```
del frame2["eastern"]
frame2.columns
```

```
Index(['year', 'state', 'pop', 'debt'], dtype='object')
```

Lưu ý: Bất kỳ sửa đổi nào đối với các cột cũng sẽ được phản ánh trong `DataFrame`. Nếu không muốn thay đổi `DataFrame`, bạn có thể tạo một bản sao bằng phương thức `copy` của `Series`.

Một dạng dữ liệu phổ biến khác là một từ điển của các từ điển, hay còn gọi là các từ điển lồng nhau

```
populations = {"Ohio": {2000: 1.5, 2001: 1.7, 2002: 3.6},
               "Nevada": {2001: 2.4, 2002: 2.9}}
```

Nếu từ điển lồng nhau được dùng để khởi tạo cho `DataFrame`, `pandas` sẽ hiểu các khóa `dict` bên ngoài là các cột và các khóa bên trong là các chỉ số hàng:

```
frame3 = pd.DataFrame(populations)
frame3
```

	Ohio	Nevada
2000	1.5	NaN
2001	1.7	2.4
2002	3.6	2.9

Bạn có thể chuyển vị `DataFrame`, nghĩa là hoán đổi hàng và cột, bằng cách sử dụng phương thức `.T` tương tự như một mảng `NumPy`:

```
frame3.T
```

	2000	2001	2002
Ohio	1.5	1.7	3.6
Nevada	NaN	2.4	2.9

Các khóa trong từ điển bên trong được kết hợp và sắp xếp để tạo thành chỉ số hàng trong kết quả. Nếu không muốn Python tự động tạo chỉ số hàng, chúng ta có thể chỉ định một chỉ số rõ ràng cho hàng:

```
pd.DataFrame(populations, index=[2001, 2002, 2003])
```

	Ohio	Nevada
2001	1.7	2.4
2002	3.6	2.9
2003	NaN	NaN

Các từ điển với giá trị là các chuỗi cũng được xử lý theo cách tương tự nếu được sử dụng để khởi tạo giá trị cho `DataFrame`

```
pdata = {"Ohio": frame3["Ohio"][:-1],
          "Nevada": frame3["Nevada"][:2]}
pd.DataFrame(pdata)
```

	Ohio	Nevada
2000	1.5	NaN
2001	1.7	2.4

Bảng ... mô tả các dạng đầu vào khả thi để tạo `DataFrame`

Bảng ... Các đầu vào dữ liệu khả thi cho hàm tạo `DataFrame`

Loại	Ghi chú
<code>dict</code> của các mảng một chiều, danh sách, <code>dict</code> , hoặc <code>Series</code>	Mỗi <code>ndarray</code> phải có cùng độ dài. Nếu một chỉ số được khởi tạo, độ dài của chỉ số phải khớp với độ dài của các mảng. Nếu không có chỉ số nào được truyền, kết quả sẽ là <code>RangeIndex(0, ..., N - 1)</code> là chỉ số của nó.
Mảng hai chiều của NumPy (<code>ndarray</code>)	Được xử lý như một ma trận dữ liệu, chuyển đổi thành một <code>DataFrame</code> với các chỉ số hàng và cột. Nếu không có chỉ số nào được truyền, chúng sẽ được gán dưới dạng số nguyên.
<code>ndarray</code> có cấu trúc hoặc bản ghi	Được xử lý tương tự như một <code>dict</code> của các <code>Series</code> .
Một <code>Series</code> khác	Chỉ số của <code>Series</code> được sử dụng làm chỉ số hàng của <code>DataFrame</code> kết quả và tên của nó được sử dụng cho tên cột của <code>DataFrame</code> .
Một <code>DataFrame</code> khác	Chỉ số của <code>DataFrame</code> được giữ lại trừ khi một chỉ số khác được truyền.

Nếu thuộc tính `index` và `columns` của `DataFrame` được đặt tên, chúng cũng sẽ được hiển thị trong bản tóm tắt:

```
frame3.index.name = "year"
frame3.columns.name = "state"
frame3
```


state	Ohio	Nevada
year		
2000	1.5	NaN
2001	1.7	2.4
2002	3.6	2.9

6.2. Chỉ số trong các cấu trúc dữ liệu cơ bản

6.2.1. Đối tượng chỉ số (Index)

Đối tượng `Index` của `pandas` chịu trách nhiệm lưu trữ các nhãn hay tên của các trục. Bất kỳ mảng hoặc chuỗi nào khác mà bạn sử dụng khi xây dựng một `Series` hoặc `DataFrame` đều được chuyển đổi nội bộ thành một `Index`:

```
obj = pd.Series(np.arange(3), index=["a", "b", "c"])
index = obj.index
index
```

```
Index(['a', 'b', 'c'], dtype='object')
```

```
index[1:]
```

```
Index(['b', 'c'], dtype='object')
```

Các đối tượng `Index` của `pandas` là **bất biến** và do đó không thể được sửa đổi bởi người dùng:

```
# index[1] = "d" # Sẽ báo lỗi TypeError
```

Tính bất biến giúp cho các đối tượng `Index` an toàn hơn để chia sẻ giữa các cấu trúc dữ liệu:

```
labels = pd.Index(np.arange(3))
labels
```

```
Index([0, 1, 2], dtype='int64')
```

```
obj2 = pd.Series([1.5, -2.5, 0], index=labels)
obj2
```

```
0    1.5
1   -2.5
2    0.0
dtype: float64
```

```
obj2.index is labels
```

```
True
```

Lưu ý: Mặc dù người dùng không cần phải lo lắng về việc các đối tượng `Index` có thể thay đổi hay không, nhưng điều đáng chú ý là một số người dùng sẽ tình cờ thay đổi một số trường trên một `Index` (như `name`). Điều này có thể thực hiện được và một số thao tác sẽ trả về một `Index` mới. Do đó, `Index` không hoàn toàn “bất biến” theo nghĩa chặt chẽ, nhưng được coi là bất biến đối với các mục đích thực tế khi các nhãn của nó không thể thay đổi.

Ngoài việc giống như một mảng, một `Index` cũng hoạt động giống như một tập hợp có kích thước cố định:

```
frame3
```

state	Ohio	Nevada
year		
2000	1.5	NaN
2001	1.7	2.4
2002	3.6	2.9

```
frame3.columns
```

```
Index(['Ohio', 'Nevada'], dtype='object', name='state')
```

```
"Ohio" in frame3.columns
```

```
True
```

```
2003 in frame3.index
```

```
False
```

Không giống như các tập hợp của Python, một `pd.Index` của `pandas` có thể chứa các nhãn trùng lặp. Việc chọn các nhãn trùng lặp sẽ chọn tất cả các lần xuất hiện của nhãn đó.

```
pd.Index(["foo", "foo", "bar", "bar"])
```

```
Index(['foo', 'foo', 'bar', 'bar'], dtype='object')
```

Mỗi `Index` có một số phương thức và thuộc tính để thực hiện các phép toán tập hợp và các tác vụ phổ biến khác. Một số phương thức và thuộc tính này được liệt kê trong Bảng ...

Bảng ... Các phương thức và thuộc tính chính của Index

Phương thức	Mô tả
<code>append</code>	Nối thêm các đối tượng <code>Index</code> bổ sung, tạo ra một <code>Index</code> mới.
<code>difference</code>	Tính toán hiệu tập hợp dưới dạng một <code>Index</code> .
<code>intersection</code>	Tính toán giao tập hợp.
<code>union</code>	Tính toán hợp tập hợp.
<code>isin</code>	Tính toán một mảng boolean cho biết liệu mỗi giá trị có nằm trong tập hợp được truyền hay không.
<code>delete</code>	Tính toán <code>Index</code> mới bằng cách xóa phần tử ở vị trí <code>i</code> .
<code>drop</code>	Tính toán <code>Index</code> mới bằng cách xóa các giá trị được truyền.
<code>insert</code>	Tính toán <code>Index</code> mới bằng cách chèn phần tử ở vị trí <code>i</code> .
<code>is_monotonic_increasing</code>	Trả về <code>True</code> nếu mỗi phần tử lớn hơn hoặc bằng phần tử trước đó.
<code>is_monotonic_decreasing</code>	Trả về <code>True</code> nếu mỗi phần tử nhỏ hơn hoặc bằng phần tử trước đó.
<code>is_unique</code>	Trả về <code>True</code> nếu <code>Index</code> không có nhãn trùng lặp.
<code>unique</code>	Tính toán mảng các nhãn duy nhất trong <code>Index</code> , trả về theo thứ tự xuất hiện.
<code>name</code>	Thuộc tính để gán tên cho một đối tượng <code>Index</code> .
<code>values</code>	Trả về <code>Index</code> dưới dạng một <code>ndarray</code> (lưu ý: sử dụng <code>Index.array</code> để lấy <code>pandas.api.extensions.ExtensionArray</code> thực tế).

6.2.2. Đánh lại chỉ số

Một phương thức quan trọng trên các đối tượng `pandas` là `reindex`, có nghĩa là tạo một đối tượng mới với dữ liệu được *điều chỉnh* cho phù hợp với các chỉ số mới. Hãy xem xét một ví dụ sau:

```
obj = pd.Series([4.5, 7.2, -5.3, 3.6], index=["d", "b", "a", "c"])
obj
```

```
d    4.5
b    7.2
a   -5.3
c    3.6
dtype: float64
```

Gọi `reindex` trên `Series` này sẽ sắp xếp lại dữ liệu theo chỉ số mới, tạo ra các giá trị bị thiếu nếu bất kỳ giá trị chỉ mục nào chưa có mặt:

```
obj2 = obj.reindex(["a", "b", "c", "d", "e", "a"])
obj2
```

```
a   -5.3
b    7.2
c    3.6
d    4.5
e    NaN
a   -5.3
dtype: float64
```

Đối với dữ liệu được sắp xếp theo thứ tự, như chuỗi thời gian, bạn đọc có thể thực hiện một số phép nội suy hoặc điền giá trị khi đánh lại chỉ số. Tùy chọn `method` cho phép chúng ta làm điều này, sử dụng một phương thức như `ffill`, viết tắt của *forward-fill*, để khởi tạo giá trị quan sát cuối cùng về phía trước:

```
obj3 = pd.Series(["blue", "purple", "yellow"], index=[0, 2, 4])
obj3
```

```
0    blue
2  purple
4  yellow
dtype: object
```

```
obj3.reindex(np.arange(6), method="ffill")
```

```
0    blue
1    blue
2  purple
3  purple
4  yellow
5  yellow
dtype: object
```

Với một `DataFrame`, `reindex` có thể thay đổi chỉ số, các cột, hoặc cả hai:

```
frame = pd.DataFrame(np.arange(9).reshape((3, 3)),
                      index=["a", "c", "d"],
                      columns=["Ohio", "Texas", "California"])
frame
```

	Ohio	Texas	California
a	0	1	2
c	3	4	5
d	6	7	8

```
frame2 = frame.reindex(index=["a", "b", "c", "d", "c"])
frame2
```

	Ohio	Texas	California
a	0.0	1.0	2.0
b	NaN	NaN	NaN
c	3.0	4.0	5.0
d	6.0	7.0	8.0
c	3.0	4.0	5.0

Các chỉ số cột có thể được đánh lại bằng từ khóa `columns`:

```
states = ["Texas", "Utah", "California"]
frame.reindex(columns=states)
```

	Texas	Utah	California
a	1	NaN	2
c	4	NaN	5
d	7	NaN	8

Các tham số tùy chọn của `reindex` được liệt kê trong Bảng ...

Bảng ... Các tham số của hàm `reindex`

Đối số	Mô tả
<code>index</code>	Chuỗi mới để sử dụng làm chỉ số (<code>Index</code> , <code>array-like</code>). Có thể là một đối tượng <code>Index</code> hoặc bất kỳ chuỗi nào khác có thể chuyển đổi thành <code>Index</code> . <code>Index</code> hỗ trợ các bản sao.
<code>method</code>	Phương pháp nội suy (điền): <code>ffill</code> điền giá trị hợp lệ cuối cùng về phía trước; <code>bfill</code> điền giá trị hợp lệ tiếp theo về phía sau. Mặc định là không điền.
<code>fill_value</code>	Giá trị thay thế để sử dụng cho các giá trị bị thiếu được giới thiệu bằng cách đánh lại chỉ mục. Mặc định là điền <code>NaN</code> .
<code>limit</code>	Khi điền tiến hoặc điền lùi, số lượng phần tử tối đa để điền.
<code>tolerance</code>	Khi điền tiến hoặc điền lùi, khoảng cách tuyệt đối tối đa mà các giá trị hợp lệ có thể được điền. Điều này chỉ áp dụng cho các chỉ mục dựa trên số.
<code>level</code>	Khớp <code>Index</code> đơn giản trên một mức của <code>MultiIndex</code> ; ngược lại chọn một tập hợp con của nó.
<code>copy</code>	Nếu <code>True</code> , luôn trả về dữ liệu mới ngay cả khi chỉ mục mới giống hệt chỉ mục cũ; nếu <code>False</code> , không sao chép dữ liệu khi chỉ số giống nhau. Mặc định là <code>True</code> .

6.2.3. Loại bỏ các chỉ số từ một trục

Loại bỏ một hoặc nhiều chỉ số từ một hàng hay một cột là khá đơn giản nếu bạn đã có một mảng chỉ số hoặc danh sách mà chúng ta muốn loại bỏ. Phương thức `drop` sẽ trả về một đối tượng mới với các giá trị được chỉ định bị xóa khỏi một hàng hay một cột

```
obj = pd.Series(np.arange(5.), index=["a", "b", "c", "d", "e"])
obj
```

```
a    0.0
b    1.0
c    2.0
d    3.0
e    4.0
dtype: float64
```

```
new_obj = obj.drop("c")
new_obj
```

```
a    0.0
b    1.0
d    3.0
e    4.0
dtype: float64
```

```
obj.drop(["d", "c"])
```

```
a    0.0
b    1.0
e    4.0
dtype: float64
```

Với `DataFrame`, các giá trị chỉ số có thể được xóa khỏi hàng hay cột:

```
data = pd.DataFrame(np.arange(16).reshape((4, 4)),
                    index=["Ohio", "Colorado", "Utah", "New York"],
                    columns=["one", "two", "three", "four"])
data
```


	one	two	three	four
Ohio	0	1	2	3
Colorado	4	5	6	7
Utah	8	9	10	11
New York	12	13	14	15

Phương thức `drop` với một chuỗi chỉ số sẽ loại bỏ các hàng dựa trên các giá trị trong chỉ số của hàng

```
data.drop(index=["Colorado", "Ohio"])
```

	one	two	three	four
Utah	8	9	10	11
New York	12	13	14	15

Bạn có thể loại bỏ các giá trị từ các cột bằng cách sử dụng tham số `axis=1` hoặc `axis="columns"`:

```
data.drop(columns=["two"])
```

	one	three	four
Ohio	0	2	3
Colorado	4	6	7
Utah	8	10	11
New York	12	14	15

```
data.drop("two", axis=1)
```

	one	three	four
Ohio	0	2	3
Colorado	4	6	7
Utah	8	10	11
New York	12	14	15

```
data.drop(["two", "four"], axis="columns")
```

	one	three
Ohio	0	2
Colorado	4	6
Utah	8	10
New York	12	14

6.2.4. Tạo chỉ số, lựa chọn và lọc dữ liệu

Tạo chỉ số cho `Series` hoạt động tương tự như tạo chỉ số cho mảng trong `NumPy`, ngoại trừ việc bạn có thể sử dụng các giá trị chỉ số của `Series` thay vì chỉ các số nguyên

```
obj = pd.Series(np.arange(4.), index=["a", "b", "c", "d"])
obj
```

```
a    0.0
b    1.0
c    2.0
d    3.0
dtype: float64
```

```
obj["b"]
```

```
np.float64(1.0)
```

```
obj[1]
```

```
np.float64(1.0)
```

```
obj[2:4]
```

```
c    2.0  
d    3.0  
dtype: float64
```

```
obj[["b", "a", "d"]]
```

```
b    1.0  
a    0.0  
d    3.0  
dtype: float64
```

```
obj[[1, 3]]
```

```
b    1.0  
d    3.0  
dtype: float64
```

```
obj[obj < 2]
```

```
a    0.0  
b    1.0  
dtype: float64
```

Lấy mảng con của `Series` với các chỉ số được chỉ định hoạt động khác với Python thông thường ở chỗ chỉ số cuối được bao gồm trong kết quả

```
obj["b":"c"]
```

```
b    1.0
c    2.0
dtype: float64
```

Việc gán giá trị trực tiếp sẽ sửa đổi phần tương ứng của `Series`:

```
obj["b":"c"] = 5
obj
```

```
a    0.0
b    5.0
c    5.0
d    3.0
dtype: float64
```

Tạo chỉ số cho một `DataFrame` là để truy xuất một hoặc nhiều cột bằng một giá trị đơn lẻ hoặc một chuỗi:

```
data = pd.DataFrame(np.arange(16).reshape((4, 4)),
                    index=["Ohio", "Colorado", "Utah", "New York"],
                    columns=["one", "two", "three", "four"])
data
```

	one	two	three	four
Ohio	0	1	2	3
Colorado	4	5	6	7
Utah	8	9	10	11
New York	12	13	14	15

```
data["two"]
```

```
Ohio    1
Colorado 5
Utah     9
New York 13
Name: two, dtype: int64
```

```
data[["three", "one"]]
```

	three	one
Ohio	2	0
Colorado	6	4
Utah	10	8
New York	14	12

Có thể lập chỉ số với các số nguyên, hoặc bằng cách sử dụng mảng kiểu logical như sau

```
data[:2]
```

	one	two	three	four
Ohio	0	1	2	3
Colorado	4	5	6	7

```
data[data["three"] > 5]
```

	one	two	three	four
Colorado	4	5	6	7
Utah	8	9	10	11
New York	12	13	14	15

Một vài ví dụ khác về tạo chỉ số cho `DataFrame` sử dụng biến logical:

```
data < 5
```

	one	two	three	four
Ohio	True	True	True	True
Colorado	True	False	False	False
Utah	False	False	False	False
New York	False	False	False	False

```
data[data < 5] = 0
data
```

	one	two	three	four
Ohio	0	0	0	0
Colorado	0	5	6	7
Utah	8	9	10	11
New York	12	13	14	15

Có thể thấy rằng cách tạo chỉ số cho `DataFrame` tương tự về mặt cú pháp với một mảng `NumPy` hai chiều.

6.2.5. Lựa chọn dữ liệu với `loc` và `iloc`

Khi tạo chỉ số cho `DataFrame` theo hàng hoặc theo cột, thư viện `pandas` có các phương thức tạo chỉ số đặc biệt là `loc` và `iloc`. Các phương thức này cho phép bạn chọn một tập hợp con các hàng và cột từ một `DataFrame` với ký hiệu giống NumPy bằng cách sử dụng tên (phương thức `loc`) hoặc số nguyên (phương thức `iloc`).

Hãy quan sát các ví dụ dưới đây

```
data.loc[["Colorado","Ohio"], ["two", "three"]]
```

	two	three
Colorado	5	6
Ohio	0	0

Nếu muốn tạo dữ liệu con với các chỉ số là số nguyên, thay vì tên hàng hoặc cột, chúng ta sử dụng `iloc`:

```
data.iloc[2, [3, 1, 1]]
```

```
four    11
two      9
two      9
Name: Utah, dtype: int64
```

```
data.iloc[2]
```

```
one      8
two      9
three    10
four     11
Name: Utah, dtype: int64
```

```
data.iloc[[1, 2], [3, 0, 1]]
```

	four	one	two
Colorado	7	0	5
Utah	11	8	9

Như vậy, có nhiều cách để chọn và sắp xếp lại dữ liệu. Đối với `DataFrame`, bảng ... tóm tắt các lựa chọn khác nhau. Bạn sẽ thấy nhiều lựa chọn trong số này hữu ích khi bạn tìm hiểu thêm về `pandas`.

Bảng ... Lựa chọn chỉ số trên DataFrame

Loại	Ghi chú
<code>df[val]</code>	Chọn một cột đơn lẻ hoặc một chuỗi các cột từ <code>DataFrame</code>
<code>df.loc[val]</code>	Chọn một hàng đơn lẻ hoặc nhiều hàng từ <code>DataFrame</code> theo tên.
<code>df.loc[:, val]</code>	Chọn một cột đơn lẻ hoặc nhiều cột theo tên.
<code>df.loc[val1, val2]</code>	Chọn cả hàng và cột theo tên.
<code>df.iloc[where]</code>	Chọn một hàng đơn lẻ hoặc nhiều hàng từ <code>DataFrame</code> theo vị trí số nguyên (từ <code>0</code> đến <code>length - 1</code>).
<code>df.iloc[:, where]</code>	Chọn một cột đơn lẻ hoặc nhiều cột theo vị trí số nguyên (từ <code>0</code> đến <code>length - 1</code>).
<code>df.iloc[where_i, where_j]</code>	Chọn cả hàng và cột theo vị trí số nguyên.
<code>df.at[label_i, label_j]</code>	Chọn một giá trị vô hướng đơn lẻ theo nhãn hàng và cột.
<code>df.iat[i, j]</code>	Chọn một giá trị vô hướng đơn lẻ theo vị trí số nguyên của hàng và cột.
<code>reindex</code> phương thức	Chọn hàng hoặc cột theo tên, tuân theo chỉ số mới.

Chúng ta sẽ quay lại việc lập chỉ số của `DataFrame` trong phần biến đổi và sắp xếp lại dữ liệu.

6.3. Tính toán trên các cấu trúc dữ liệu cơ bản

6.3.1. Phép toán số học và căn chỉnh dữ liệu

`pandas` có thể giúp bạn làm việc với dữ liệu từ các nguồn không đồng nhất hoặc khác nhau. Ví dụ, khi bạn cộng các đối tượng lại với nhau, nếu bất kỳ cặp chỉ số nào không giống nhau, chỉ số tương ứng trong kết quả sẽ là hợp của các cặp chỉ số. Hãy xem một ví dụ sau


```
s1 = pd.Series([7.3, -2.5, 3.4, 1.5], index=["a", "c", "d", "e"])
s2 = pd.Series([-2.1, 3.6, -1.5, 4, 3.1],
               index=["a", "c", "e", "f", "g"])

s1
```

```
a    7.3
c   -2.5
d    3.4
e    1.5
dtype: float64
```

```
s2
```

```
a   -2.1
c    3.6
e   -1.5
f    4.0
g    3.1
dtype: float64
```

Cộng hai `Series` với nhau sẽ tạo ra:

```
s1 + s2
```

```
a    5.2
c    1.1
d    NaN
e    0.0
f    NaN
g    NaN
dtype: float64
```

`pandas` tự động căn chỉnh và giới thiệu các giá trị `NaN` ở các vị trí tên không trùng khớp. Trong trường hợp `DataFrame`, việc căn chỉnh được thực hiện trên cả hàng và cột:

```
df1 = pd.DataFrame(np.arange(9.).reshape((3, 3)), columns=list("bcd"),
                   index=["Ohio", "Texas", "Colorado"])
df2 = pd.DataFrame(np.arange(12.).reshape((4, 3)), columns=list("bde"),
                   index=["Utah", "Ohio", "Texas", "Oregon"])

df1
```

	b	c	d
Ohio	0.0	1.0	2.0
Texas	3.0	4.0	5.0
Colorado	6.0	7.0	8.0

df2

	b	d	e
Utah	0.0	1.0	2.0
Ohio	3.0	4.0	5.0
Texas	6.0	7.0	8.0
Oregon	9.0	10.0	11.0

Cộng hai `DataFrame` lại với nhau sẽ trả về một `DataFrame` có chỉ số cột là hợp của hai chỉ số cột, đồng thời chỉ số hàng là hợp của hai chỉ số hàng:

df1 + df2

	b	c	d	e
Colorado	NaN	NaN	NaN	NaN
Ohio	3.0	NaN	6.0	NaN
Oregon	NaN	NaN	NaN	NaN
Texas	9.0	NaN	12.0	NaN
Utah	NaN	NaN	NaN	NaN

Vì các cột `"c"` và `"e"` không được tìm thấy trong cả hai `DataFrame`, do đó giá trị xuất hiện trong kết quả đều là `NaN`. Điều tương tự với các hàng có tên không xuất hiện ở cả hai `DataFrame`.

Nếu bạn cộng các đối tượng `DataFrame` không có tên cột hoặc hàng chung, kết quả có tất cả các giá trị là `NaN`:

```
df1 = pd.DataFrame({"A": [1, 2]})
df2 = pd.DataFrame({"B": [3, 4]})
df1
```

	A
0	1
1	2

df2

	B
0	3
1	4

df1 + df2

	A	B
0	NaN	NaN
1	NaN	NaN

Để tránh trường hợp gặp các giá trị toàn `NaN`, khi thực hiện phép toán trên hai hay nhiều `DataFrame`, chúng ta có thể sử dụng phương thức `add` đi kèm với tham số `fill_value` để khởi tạo giá trị cho kết quả. Hãy quan sát cách sử dụng phương thức `add` trong các ví dụ dưới đây:

```
df1 = pd.DataFrame(np.arange(12).reshape((3, 4)),
                    columns=list("abcd"))
df2 = pd.DataFrame(np.arange(20).reshape((4, 5)),
                    columns=list("abcde"))
df2.loc[1, "b"] = np.nan
df1
```

	a	b	c	d
0	0.0	1.0	2.0	3.0
1	4.0	5.0	6.0	7.0
2	8.0	9.0	10.0	11.0

df2

	a	b	c	d	e
0	0.0	1.0	2.0	3.0	4.0
1	5.0	NaN	7.0	8.0	9.0
2	10.0	11.0	12.0	13.0	14.0
3	15.0	16.0	17.0	18.0	19.0

Nếu cộng df1 và df2 lại với nhau sẽ dẫn đến các giá trị NaN ở những vị trí không trùng lặp:

df1 + df2

	a	b	c	d	e
0	0.0	2.0	4.0	6.0	NaN
1	9.0	NaN	13.0	15.0	NaN
2	18.0	20.0	22.0	24.0	NaN
3	NaN	NaN	NaN	NaN	NaN

Sử dụng phương thức add trên df1 đối với df2 và tham số fill_value là giá trị thay thế cho những giá trị không quan sát được

```
df1.add(df2, fill_value=0)
```

	a	b	c	d	e
0	0.0	2.0	4.0	6.0	4.0
1	9.0	5.0	13.0	15.0	9.0
2	18.0	20.0	22.0	24.0	14.0
3	15.0	16.0	17.0	18.0	19.0

Bảng ... liệt kê danh sách các phương thức số học của `Series` và `DataFrame`. Mỗi phương thức đều có phương thức khác đi kèm, với tên gọi được thêm chữ `r` vào trước. Khi có `r` đứng trước, phương thức số học sẽ được hiểu theo thứ tự từ phải qua trái, nghĩa là `A.radd(B)` sẽ tương đương với `B + A`. Mặc dù nhiều phương thức số học có tính chất giao hoán, nhưng không hẳn lúc nào phương thức đi kèm cũng cho kết quả giống nhau, nguyên nhân là do `pandas` có nhiều tùy chỉnh khi thực hiện tác phương thức này. Chúng ta sẽ thảo luận kỹ hơn trong phần sau của chương.

Bảng ... Các phương thức số học thực hiện trên các `DataFrame`

Phương thức	Mô tả
<code>add</code> , <code>radd</code>	Phương thức cộng (+)
<code>sub</code> , <code>rsub</code>	Phương thức trừ (-)
<code>div</code> , <code>rdiv</code>	Phương thức chia (/)
<code>floordiv</code> , <code>rfloordiv</code>	Phương thức chia lấy phần nguyên (//)
<code>mul</code> , <code>rmul</code>	Phương thức nhân (*)
<code>pow</code> , <code>rpow</code>	Phương thức lũy thừa (**)

6.3.2. Phép toán giữa DataFrame và Series

Tương tự như các mảng `NumPy` có kích thước khác nhau, các phép toán số học giữa `DataFrame` và `Series` cũng có thể được thực hiện một cách tương tự

```
arr = np.arange(12).reshape((3, 4))
arr
```

```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

```
arr[0]
```

```
array([0, 1, 2, 3])
```

```
arr - arr[0]
```

```
array([[0, 0, 0, 0],
       [4, 4, 4, 4],
       [8, 8, 8, 8]])
```

Quan sát kết quả, bạn đọc có thể thấy rằng khi chúng ta trừ `arr[0]` khỏi `arr`, phép trừ được thực hiện trên từng hàng một. Cơ chế này được gọi là **lan truyền**, hay broadcasting. Các phép toán giữa `DataFrame` và `Series` cũng tương tự:

```
frame = pd.DataFrame(np.arange(12.).reshape((4, 3)),
                      columns=list("bde"),
                      index=["Utah", "Ohio", "Texas", "Oregon"])
series = frame.iloc[0]
frame
```

	b	d	e
Utah	0.0	1.0	2.0
Ohio	3.0	4.0	5.0
Texas	6.0	7.0	8.0
Oregon	9.0	10.0	11.0

```
series
```

```
b    0.0
d    1.0
e    2.0
Name: Utah, dtype: float64
```

Theo mặc định, phép toán số học giữa `DataFrame` và `Series` khớp chỉ số của `Series` với các cột của `DataFrame`, lan truyền từ trên xuống qua các hàng:

```
frame - series
```

	b	d	e
Utah	0.0	0.0	0.0
Ohio	3.0	3.0	3.0
Texas	6.0	6.0	6.0
Oregon	9.0	9.0	9.0

Nếu một giá trị chỉ số không được tìm thấy trong các cột của `DataFrame` hoặc trong chỉ số của `Series`, các đối tượng sẽ được đánh lại chỉ số để tạo thành hợp của các chỉ số:

```
series2 = pd.Series(np.arange(3), index=["b", "e", "f"])
series2
```

```
b    0
e    1
f    2
dtype: int64
```

```
frame + series2
```

	b	d	e	f
Utah	0.0	NaN	3.0	NaN
Ohio	3.0	NaN	6.0	NaN
Texas	6.0	NaN	9.0	NaN
Oregon	9.0	NaN	12.0	NaN

Nếu bạn muốn lan truyền qua các cột, và chỉ số khởi với các hàng, bạn phải sử dụng một trong các phương thức số học đã nêu ở trên và chỉ định `axis="index"` hoặc `axis=0`:

```
series3 = frame["d"]
frame
```

	b	d	e
Utah	0.0	1.0	2.0
Ohio	3.0	4.0	5.0
Texas	6.0	7.0	8.0
Oregon	9.0	10.0	11.0

```
series3
```

```
Utah      1.0
Ohio      4.0
Texas     7.0
Oregon    10.0
Name: d, dtype: float64
```

```
frame.sub(series3, axis="index")
```


	b	d	e
Utah	-1.0	0.0	1.0
Ohio	-1.0	0.0	1.0
Texas	-1.0	0.0	1.0
Oregon	-1.0	0.0	1.0

Phương thức `sub` ở trên có nghĩa là chúng ta đang trừ `series3` khỏi mỗi hàng của `frame`, nơi chỉ số hàng của `frame` khớp với chỉ số của `series3`.

6.3.3. Phương thức `apply`

Một thao tác thường gặp là áp dụng một hàm trên các mảng một chiều là các cột hoặc các hàng của một `DataFrame`. Bạn đọc sử dụng phương thức `apply` của `DataFrame` để thực hiện những tính toán như vậy. Giả sử có một `DataFrame` có tên `frame` như sau

```
frame = pd.DataFrame(np.random.standard_normal((4, 3)),
                      columns=list("bde"),
                      index=["Utah", "Ohio", "Texas", "Oregon"])
frame
```

	b	d	e
Utah	-1.784738	-0.527362	-1.385733
Ohio	-0.506815	-0.501790	-0.133087
Texas	0.514935	1.068865	-0.219565
Oregon	0.125756	0.467556	-0.338263

Nếu chúng ta muốn tính toán hiệu giữa giá trị lớn nhất và nhỏ nhất của mỗi cột trong `frame`, chúng ta khai báo hàm cần thực hiện và sau đó sử dụng phương thức `apply` như sau

```
def f1(x):
    return x.max() - x.min()

frame.apply(f1)
```

```
b    2.299673
d    1.596227
e    1.252646
dtype: float64
```

Nếu bạn sử dụng tham số `axis="columns"` trong phương thức `apply`, hàm sẽ thực hiện tính toán trên các hàng thay vì trên các cột.

```
frame.apply(f1, axis="columns")
```

```
Utah    1.257375
Ohio    0.373728
Texas   1.288430
Oregon  0.805819
dtype: float64
```

Thay vì tự khai báo hàm số, bạn đọc có thể sử dụng các hàm thống kê phổ biến như `sum`, `mean`, ...

```
frame.apply(sum, axis="columns")
```

```
Utah    -3.697833
Ohio    -1.141691
Texas    1.364235
Oregon   0.255050
dtype: float64
```

Hàm được sử dụng trong `apply` không nhất thiết phải trả về một giá trị, mà kết quả củ

Cell In[119], line 1

Hàm được sử dụng trong `apply` không nhất thiết phải trả về một giá trị, mà kết quả

`SyntaxError`: invalid syntax

```
def f2(x):
    return pd.Series([x.min(), x.max()], index=["min", "max"])
frame.apply(f2)
```

	b	d	e
min	-1.718820	-1.268898	-0.119529
max	2.467343	1.818112	0.750917

Có thể thấy rằng khi hàm trả kết quả là một `Series`, phương thức `apply` trên một `DataFrame` sẽ cho kết quả là một mảng nhiều chiều.

6.3.4. Sắp xếp dữ liệu

Sắp xếp một tập dữ liệu theo một số tiêu chí là một yêu cầu quan trọng trong xử lý dữ liệu. Để sắp xếp theo chỉ số hàng hoặc chỉ số cột, có thể sử dụng phương thức `sort_index`, phương thức này trả về một đối tượng mới được sắp xếp:

```
obj = pd.Series(np.arange(4), index=["d", "a", "b", "c"])
obj.sort_index()
```

Với một `DataFrame`, bạn có thể sắp xếp theo chỉ số theo hàng hoặc chỉ số theo cột:

```
frame = pd.DataFrame(np.arange(8).reshape((2, 4)),
                     index=["three", "one"],
                     columns=["d", "a", "b", "c"])
frame.sort_index()
```

```
frame.sort_index(axis="columns")
```

Dữ liệu được sắp xếp theo thứ tự tăng dần theo mặc định, nhưng có thể được sắp xếp theo thứ tự giảm dần bằng cách thêm tham số `ascending = False`

```
frame.sort_index(axis="columns", ascending=False)
```

Để sắp xếp một `Series` theo các giá trị của nó, bạn đọc có thể sử dụng phương thức `sort_values`:

```
obj = pd.Series([4, 7, -3, 2])
obj.sort_values()
```

Các giá trị không quan sát được sẽ được sắp xếp về cuối `Series`:

```
obj = pd.Series([4, np.nan, 7, np.nan, -3, 2])
obj.sort_values()
```

Nếu muốn sắp xếp các giá trị `NaN` lên đầu, có thể sử dụng tham số `na_position`:

```
obj.sort_values(na_position="first")
```

Khi sắp xếp một `DataFrame`, bạn có thể sử dụng dữ liệu trong một hoặc nhiều cột làm khóa sắp xếp. Để làm điều này, hãy tạo một hoặc nhiều tên cột cho tham số `by` của phương thức `sort_values`:

```
frame = pd.DataFrame({"b": [4, 7, -3, 2], "a": [0, 1, 0, 1]})
frame
```

```
frame.sort_values(by="b")
```

Có thể sử dụng nhiều cột với tham số `by`:

```
frame.sort_values(by=["a", "b"])
```

Phương thức `rank` có thể được sử dụng cho `Series` và `DataFrame` để tạo ra các `Series` và `DataFrame`. Khi không khai báo thêm tham số, phương thức `rank` mặc định tham số `method` nhận giá trị `average`, nghĩa là các giá trị bằng nhau sẽ có thứ hạng bằng giá trị trung bình:

```
obj = pd.Series([7, -5, 7, 4, 2, 0, 4])
obj.rank()
```

```
0    6.5
1    1.0
2    6.5
3    4.5
4    3.0
5    2.0
6    4.5
dtype: float64
```

Chúng ta cũng có thể sử dụng các giá trị khác cho tham số `method`, chẳng như khi muốn các thứ hạng được gán theo thứ tự được quan sát trong dữ liệu:

```
obj.rank(method="first")
```

Có thể thấy rằng, thay vì sử dụng thứ hạng trung bình 6.5 cho các chỉ số 0 và 2, thứ hạng được đổi thành 6 và 7 vì chỉ số 0 đứng trước chỉ số 2 trong dữ liệu. Bạn đọc cũng có thể xếp hạng theo thứ tự giảm dần, cũng như tăng dần:

```
# Gán thứ hạng ràng buộc cho giá trị lớn nhất trong nhóm
obj.rank(ascending=False, method="max")
```

Phương thức `rank` trên `DataFrame` sẽ được thực hiện đồng thời trên các hàng hoặc các cột:

```
frame = pd.DataFrame({"b": [4.3, 7, -3, 2], "a": [0, 1, 0, 1],
                      "c": [-2, 5, 8, -2.5]})
frame
```

	b	a	c
0	4.3	0	-2.0
1	7.0	1	5.0
2	-3.0	0	8.0
3	2.0	1	-2.5

```
frame.rank(axis="columns")
```

	b	a	c
0	3.0	2.0	1.0
1	3.0	1.0	2.0
2	1.0	2.0	3.0
3	3.0	2.0	1.0

Bảng ... liệt kê các giá trị có thể được sử dụng cho tham số `method` của phương thức `rank`

Bảng ... Các phương thức xử lý ràng buộc với `rank`

Giá trị	Mô tả
<code>"average"</code>	Gán thứ hạng trung bình cho mỗi giá trị trong nhóm ràng buộc.
<code>"min"</code>	Sử dụng thứ hạng nhỏ nhất trong toàn bộ nhóm.
<code>"max"</code>	Sử dụng thứ hạng lớn nhất trong toàn bộ nhóm.
<code>"first"</code>	Gán các thứ hạng theo thứ tự các giá trị xuất hiện trong dữ liệu.
<code>"dense"</code>	Giống như <code>"min"</code> , nhưng thứ hạng luôn tăng thêm 1 giữa các nhóm thay vì số lượng các phần tử bằng nhau trong một nhóm.

Tất cả các ví dụ chúng ta xem xét từ đầu chương đều có tên hay chỉ số duy nhất. Mặc dù đa số các hàm trong `pandas` yêu cầu các tên phải là duy nhất, nhưng điều này không bắt buộc với tất cả mọi hàm. Hãy xem xét một `Series` với các chỉ số trùng lặp như sau

```
import numpy as np
obj = pd.Series(np.arange(5), index=["a", "a", "b", "b", "c"])
obj
```

```
a    0
a    1
b    2
b    3
c    4
dtype: int64
```

Phương thức `is_unique` của chỉ số cho chúng ta biết liệu tên của các chỉ số có duy nhất hay không:

```
obj.index.is_unique
```

```
False
```

Nếu một tên bị lặp lại nhiều lần được gọi từ `DataFrame`, kết quả sẽ trả về một `Series`. Còn nếu tên là duy nhất một giá trị sẽ được trả về:

```
obj["a"]
```

```
a    0
a    1
dtype: int64
```

```
obj["c"]
```

Logic tương tự cũng áp dụng cho việc lập chỉ số các hàng hoặc các cột trong một `DataFrame`:

```
df = pd.DataFrame(np.random.standard_normal((5, 3)),
                  index=["a", "a", "b", "b", "c"])
df
```

```
df.loc["b"]
```

```
df.loc["c"]
```

6.4. Các phương thức nhằm tóm tắt và tính toán thống kê mô tả

Các đối tượng của `pandas` được trang bị một tập hợp các phương thức toán học và thống kê. So với các phương thức tương đương của mảng trong `NumPy`, các phương thức này được xây

dùng để **tự động loại trừ dữ liệu không quan sát được** (`NA` hoặc `NaN`):

```
df = pd.DataFrame([[1.4, np.nan], [7.1, -4.5],  
                  [np.nan, np.nan], [0.75, -1.3]],  
                  index=["a", "b", "c", "d"],  
                  columns=["one", "two"])  
  
df
```

	one	two
a	1.40	NaN
b	7.10	-4.5
c	NaN	NaN
d	0.75	-1.3

Phương thức `sum` của `DataFrame` sẽ trả về một `Series` chứa tổng các cột, bỏ qua các giá trị không quan sát được

```
df.sum()
```

```
one    9.25  
two   -5.80  
dtype: float64
```

Sử dụng tham số `axis="columns"` hoặc `axis=1` để tính tổng trên các hàng thay vì cột:

```
df.sum(axis="columns")
```

Các giá trị không quan sát được bị bỏ qua trừ khi toàn bộ giá trị trong một hàng hoặc một cột đều là không quan sát được. Nếu chúng ta muốn tính toán đến cả các giá trị không quan sát được, mặc dù hiếm khi xảy ra, thì có thể sử dụng tham số tùy `skipna`:

```
df.sum(axis="index", skipna=False)
```



```
one    NaN
two    NaN
dtype: float64
```

```
df.sum(axis="columns", skipna=False)
```

Các phương thức `idxmin` và `idxmax` trả về các chỉ số nơi các giá trị tối thiểu hoặc tối đa đạt được:

```
df.idxmax()
```

```
df.idxmin(axis="columns")
```

Một phương thức hữu ích khác cũng thường được sử dụng là `describe`:

```
df.describe()
```

	one	two
count	3.000000	2.000000
mean	3.083333	-2.900000
std	3.493685	2.262742
min	0.750000	-4.500000
25%	1.075000	-3.700000
50%	1.400000	-2.900000
75%	4.250000	-2.100000
max	7.100000	-1.300000

Trên dữ liệu không phải dạng số, `describe` tạo ra các thống kê tóm tắt cho biến kiểu phân loại:

```
obj = pd.Series(["a", "a", "b", "c"] * 4)
obj.describe()
```

```
count      16
unique      3
top         a
freq        8
dtype: object
```

Bảng ... liệt kê một số thống kê mô tả và phương thức liên quan.

Bảng 5.7. Các tùy chọn thống kê mô tả và tóm tắt

Phương thức	Mô tả
<code>count</code>	Số lượng các giá trị không phải NA.
<code>describe</code>	Tính toán một tập hợp các thống kê tóm tắt cho <code>Series</code> hoặc cho mỗi cột <code>DataFrame</code> .
<code>min</code> , <code>max</code>	Tính toán các giá trị tối thiểu và tối đa.
<code>argmin</code> , <code>argmax</code>	Tính toán các vị trí chỉ số (số nguyên) mà tại đó các giá trị tối thiểu hoặc tối đa đạt được, tương ứng.
<code>idxmin</code> , <code>idxmax</code>	Tính toán các tên của chỉ số mà tại đó các giá trị tối thiểu hoặc tối đa đạt được, tương ứng.
<code>quantile</code>	Tính toán phân vị.
<code>sum</code>	Tổng các giá trị.
<code>mean</code>	Trung bình của các giá trị.
<code>median</code>	Trung vị số học (phân vị 50%) của các giá trị.
<code>mad</code>	Độ lệch tuyệt đối so với giá trị trung bình.
<code>prod</code>	Tích của tất cả các giá trị.
<code>var</code>	Phương sai của các giá trị.
<code>std</code>	Độ lệch chuẩn.
<code>skew</code>	Độ nghiêng mẫu (mô men bậc ba) của các giá trị.
<code>kurt</code>	Độ nhọn mẫu (mô men bậc bốn) của các giá trị.
<code>cumsum</code>	Tổng tích lũy của các giá trị.
<code>cummin</code> , <code>cummax</code>	Giá trị tối thiểu hoặc tối đa tích lũy của các giá trị, tương ứng.
<code>cumprod</code>	Tích tích lũy của các giá trị.
<code>diff</code>	Tính toán sai số bậc nhất (cho chuỗi thời gian).

Phương thức	Mô tả
<code>pct_change</code>	Tính toán thay đổi phần trăm.

Một số thống kê tóm tắt, như hiệp phương sai và hệ số tương quan, được tính toán từ hai hay nhiều `Series`. Bạn đọc có thể tính toán các hệ số này bằng cách sử dụng phương thức `cov` và `corr` từ một `Series` tới một `Series` khác. Nguyên tắc tính toán là bỏ qua các giá trị không quan sát được và chỉ số được căn chỉnh cho phù hợp:

```
frame = pd.DataFrame({"b": [4.3, 7, -3, 2], "a": [0, np.nan, 0, 1],
                      "c": [-2, 5, 8, -2.5]})

frame["a"].cov(frame["b"])
```

```
np.float64(0.45)
```

Khi các phương thức `corr` và `cov` sử dụng trên `DataFrame` sẽ trả về một ma trận tương quan hoặc hiệp phương sai đầy đủ dưới dạng `DataFrame`

```
frame.cov()
```

	b	a	c
b	17.989167	0.450000	-8.162500
a	0.450000	0.333333	-1.833333
c	-8.162500	-1.833333	27.062500

```
frame.corr()
```

	b	a	c
b	1.000000	0.208832	-0.369942
a	0.208832	1.000000	-0.536107
c	-0.369942	-0.536107	1.000000

Như vậy chương sách này đã giới thiệu các đối tượng `Series` và `DataFrame` của pandas và các công cụ cơ bản để làm việc với dữ liệu được lưu trong các đối tượng này. Trong các chương tiếp theo, bạn sẽ đi sâu hơn vào các công cụ phân tích và thao tác dữ liệu.